

FrankAie-hub · 操作文件合輯

LYCANDER AI RESEARCH HUB —— 作業指南 + GitHub 操作指南合併本 由 LYCANDER GROUP 維護 · 產生日期：2026-06-22

目錄

1. [專案總覽 \(README\)](#)
 2. [作業指南 · HUB 操作手冊](#)
 3. [GitHub 操作指南](#)
-

Part 1 · 專案總覽 (README)

FrankAie-hub · LYCANDER AI RESEARCH HUB

專業 AI 角色技能庫 (marketplace)。每個 skill 都是一位資深領域專家 AI，依「搜集市場資訊 → 驗證 → 開發 / 設計」三階段 SOP，獨立發展該領域的專業技能。

由 LYCANDER GROUP 維護。本 repo 同時是一個 Claude 外掛 marketplace，裡面收錄 `frankaie-research-hub` plugin，plugin 內含多個專家角色 skill。

編制：100 位 AI 角色

30 位專家 (6 團隊) + 70 位 AI 消費者 (4 市場)。完整名冊見 `plugins/frankaie-research-hub/ROSTER.md`。

旗艦單一專家

角色 skill	定位 (對標)	三階段 SOP
<code>electronics-engineer</code> 資深電子工程師	Anker 級	搜集市場資訊 → 驗證產品功能 → 開發對應產品
<code>animation-designer</code> 資深動畫 / 視覺設計師	Native Union 級	搜集市場資訊 → 驗證視覺轉化率 → 設計對應圖像及影像

6 個專家團隊 (每個 = 5 位具名專家, 共 30 人)

`electronics-team` (電子工程)、`mechanical-team` (機構設計)、`visual-team` (視覺設計)、`marketing-team` (行銷企劃)、`operations-team` (營運執行)、`cx-team` (售後客服)。

4 個 AI 消費者市場小組 (共 70 人)

`consumers-taiwan` (22)、`consumers-usa` (18)、`consumers-japan` (15)、`consumers-europe` (15)。用來在「驗證」階段召集虛擬焦點小組, 對產品/定價/文案/視覺給購買意願評分與改善建議。

要再擴編, 只要在 `plugins/frankaie-research-hub/skills/` 下新增資料夾 + `SKILL.md` 即可。

安裝方式 (在 Claude 中)

1. 把本 repo push 上 GitHub (`Lycander168/FrankAie-hub`) 。
2. 加入 marketplace :

```
/plugin marketplace add Lycander168/FrankAie-hub
```

1. 安裝 plugin :

```
/plugin install frankaie-research-hub@frankaie-hub
```

1. 重啟 session 後, 兩個專家角色 skill 就會出現在 skill 清單, 依描述關鍵字自動觸發。

Repo 設為 Public 最省事; 若設 Private, 安裝時需設定 GitHub 認證。

目錄結構

```
FrankAie-hub/
├── .claude-plugin/
│   └── marketplace.json      ← marketplace 定義 (指向 plugin)
├── README.md                ← 你正在看的這份
└── plugins/
    ├── frankaie-research-hub/
    │   ├── .claude-plugin/
    │   │   └── plugin.json    ← plugin 基本資料
    │   ├── README.md         ← plugin 說明 + 新增角色教學
    │   └── skills/
    │       ├── electronics-engineer/
    │       │   └── SKILL.md    ← 資深電子工程師 AI
    │       └── animation-designer/
    │           └── SKILL.md     ← 資深動畫 / 視覺設計師 AI
```

新增一位專家角色 (快速版)

1. 在 `plugins/frankaie-research-hub/skills/` 新增資料夾，例如 `supply-chain-expert/`。
2. 裡面放 `SKILL.md`，frontmatter 至少要有 `name` 與 `description` (`description` 寫清楚觸發關鍵字)。
3. 內文建議沿用「Persona → 三階段 SOP → 輸出範本 → 協作介面 → 啟動方式」結構。
4. `commit`、`push`，重新 `/plugin marketplace update frankaie-hub` 即可。

詳見 `plugins/frankaie-research-hub/README.md`。

維護者文件

- 作業指南 (HUB 操作手冊)：`docs/OPERATIONS_GUIDE.md` —— 怎麼用這 100 位角色 + 工具，把一個專案從「想法」跑到「上市與售後」的標準作業流程 (含決策樹、7 站動線、實戰範例)。
 - GitHub 操作指南：`docs/GITHUB_GUIDE.md` —— 從 clone、開分支、commit、push 到開 PR 與更新 marketplace 的完整照抄指令，適合第一次接觸 Git 的維護者。
-

Part 2 · 作業指南 (HUB 操作手冊)

作業指南 · FrankAie-hub (HUB 操作手冊)

這份是 實際使用 LYCANDER AI RESEARCH HUB 的標準作業流程 (SOP) 。教你怎麼把這 100 位 AI 角色 + 工具，從「一個想法」一路跑到「上市與售後」。想學 Git / 推送程式碼，請看另一份 docs/GITHUB_GUIDE.md 。由 LYCANDER GROUP 維護。

0. 三句話搞懂這個 HUB

1. 專家負責把產品做出來、賣出去、服務好 (30 位 · 6 團隊) 。
2. AI 消費者負責「投錢前先告訴你會不會買」 (70 位 · 4 市場) 。
3. 總機 (hub-router) 負責分流排序、工具型 skill 負責把具體產出做出來。

核心心法 (所有角色共用)：搜集市場資訊 → 驗證 → 開發 / 設計 三階段 SOP，逐段推進、逐段確認、不臆造。

1. 我該怎麼開始？（決策樹）

你的需求是.....

- ├ 一個完整專案 / 跨多領域 / 不知道該找誰
 - 先找「hub-router」，它會產出「專案作戰計畫」與分工表
- ├ 只要一位專家快速判斷（單兵）
 - 直接找旗艦專家：
 - 電子→ electronics-engineer | 視覺→ animation-designer
 - 財務→ financial-analyst | 法務→ legal-ip
- ├ 要一個領域「完整分工協作」
 - 找對應團隊：electronics / mechanical / visual / marketing / operations / cx -team
- ├ 要驗證「會不會買」
 - 找 consumers-taiwan / usa / japan / europe
- ├ 要做出一個具體產出物（listing / hook / 定價表...）
 - 直接找對應「工具型 skill」（見第 5 節）

口訣：大需求先過總機；單一任務直接點對的人。

2. 標準作業流程：新品從 0 到上市

這是 HUB 的主動線（與 hub-router 一致），共 7 站：

站	階段	召集	主要產出
1	立項	financial-analyst + consumers-* + competitor-comparison	算帳 + 需求驗證 + 競品差異
2	開發	electronics-team + mechanical-team (並行) → legal-ip	Spec / BOM / 機構 + 專利迴避
3	認證量產	electronics-team (認證) + operations-team	認證計畫 + 採購/量產
4	上市素材	visual-team + marketing-team + 工具群	KV/包裝 + 行銷 + hook/listing/seo
5	上市前驗證	consumers-*	素材/定價購買意願評分 (1-5)
6	上架投放	marketing-team + operations-team	投放 + 出貨
7	售後迭代	cx-team + review-miner	客訴/口碑 → 回饋到第 1 站

這是一個循環：第 7 站的回饋會餵回第 1 站，啟動下一輪迭代。

3. 每一站怎麼跑：三階段 SOP

不論召集誰，都會走同一套三階段。每段都要先確認再往下：

階段	做什麼	結束條件 (Gate)
① 搜集市場資訊	蒐集現況、競品、數據、限制條件	資料夠不夠下判斷？缺的有沒有標註？
② 驗證	用 consumers-* / 測試 / 指標檢驗假設	假設站得住腳嗎？購買意願 / 數據過關嗎？
③ 開發 / 設計	產出實際成果 (spec / 文案 / 視覺 / 計畫)	產出可被檢驗、可被交接嗎？

防臆造守則（最重要）：任何主張都要能回溯來源或測試指標；沒有依據的，標註「待補」而不是編一個。缺資料時，角色會主動列出「需要你先補的資訊」。

4. 實戰範例：開發並上架一顆 65W GaN 充電器

- Step 0 使用者：「我想開發並上架一顆 65W GaN 充電器」
→ 觸發 hub-router, 產出「專案作戰計畫」
- Step 1 立項
financial-analyst → 成本/毛利/回本試算
consumers-usa/taiwan → 對「65W GaN」概念打購買意願分
competitor-comparison → vs Anker/UGREEN 差異化定位
- Step 2 開發（並行）
electronics-team → 電源架構/快充協議/安全
mechanical-team → 外觀/散熱/模具
legal-ip → 專利迴避 FT0、認證紅線提醒
- Step 3 認證量產
electronics-team(認證) → UL/CE/BSMI 計畫
operations-team → 採購/打樣/量產排程
- Step 4 上市素材
visual-team → KV / 包裝
marketing-team → 賣點訊息 / 通路策略
工具群 → ad-hook-writer / listing-optimizer /
seo-keyword-expander / copy-humanizer
- Step 5 上市前驗證
consumers-* → 對 listing / 定價 / 主視覺打分, 回修
- Step 6 上架投放 + 出貨
marketing-team + operations-team
- Step 7 售後
cx-team + review-miner → 痛點回饋 → 回到 Step 1 做 v2
-

5. 工具型 skill 速查（要具體產出時直接點）

想要的產出	用這個 skill
轉換導向 Listing（標題/五點/A+/後台關鍵字）	<code>listing-optimizer</code>
廣告前三秒 hook + 短影音開場分鏡	<code>ad-hook-writer</code>
把生硬文案改自然、去 AI 味	<code>copy-humanizer</code>
從雜亂評論萃取痛點/賣點	<code>review-miner</code>
定價/毛利/break-even 試算	<code>pricing-calculator</code>
EDM 自動化序列	<code>email-sequence-builder</code>
競品加權比較 + 差異化定位	<code>competitor-comparison</code>
分群分意圖的 SEO 關鍵字地圖	<code>seo-keyword-expander</code>

6. 作業守則（黃金 6 條）

1. 大需求先過總機 —— 不確定找誰就先 `hub-router`，避免重工漏接。
2. 逐階段確認 —— 三階段 SOP 每段有 Gate，沒過不硬推下一段。
3. 凡主張必有據 —— 防臆造優先；缺資料標「待補」並列出要補什麼。
4. 投錢前先驗證 —— 重要決策（概念/定價/素材）一律過 `consumers-*` 打分。
5. 單一任務不繞路 —— 只要一個 hook / 一張定價表，直接點工具型 skill。
6. 產出要可交接 —— 用各角色的表格化輸出範本，方便下一棒接手。

7. 召集語句範例（直接照抄）

- 整案規劃：「幫我規劃開發並上架一顆 65W GaN 充電器的完整流程，召集需要的團隊。」
- 單兵諮詢：「以資深電子工程師角度，幫我快速判斷這顆充電器的快充架構可行性。」
- 市場驗證：「召集美國消費者小組，對這個定價 \$39.99 與主視覺打購買意願分。」
- 具體產出：「用 `listing-optimizer` 幫這顆充電器寫 Amazon Listing。」

8. 連接 GitHub：把變更同步上去

HUB 的角色與 skill 都存在 `Lycander168/FrankAie-hub` 這個 GitHub repo。用 HUB 做事（前面 7 節）跟同步到 GitHub 是同一條動線的兩端：


```
改 / 新增角色或文件 (本機)  
→ commit → push (GitHub)  
→ /plugin marketplace update (Claude 端生效)
```

A. 第一次連接 (安裝 marketplace)

在 Claude 中執行，連到本 repo：

```
/plugin marketplace add Lycander168/FrankAie-hub  
/plugin install frankaie-research-hub@frankaie-hub
```

Repo 設為 Public 最省事；若為 Private，安裝時需設定 GitHub 認證。

B. 改完角色 / 文件後，同步回 GitHub

完整 Git 指令 (clone、開分支、commit、push、開 PR) 見 [docs/GITHUB_GUIDE.md](#)。最短路徑：

```
git checkout -b feat/你的變更  
git add .  
git commit -m "新增 XX 角色 (vX.Y.0)"  
git push -u origin feat/你的變更  
# 在 GitHub 開 PR → 審查 → 合併進 main
```

C. 讓變更在 Claude 端生效

合併進 `main` 後，使用者端更新即可看到新角色 / 文件：

```
/plugin marketplace update frankaie-hub
```

一句話：HUB 產出 → Git 同步 → marketplace update，三步走完才算真正上線。詳細 Git 操作一律以 [docs/GITHUB_GUIDE.md](#) 為準。

由 LYCANDER GROUP 維護 · 有問題請開 issue 或聯絡 service@lycander.tw

Part 3 · GitHub 操作指南

GitHub 操作指南 · FrankAie-hub

本指南帶你從零開始，學會維護 `Lycander168/FrankAie-hub` 這個 repo 所需要的所有 GitHub 操作：取得程式碼、修改、提交、推送、開 PR，以及更新 Claude marketplace。由 LYCANDER GROUP 維護。

適用對象：第一次接觸 Git / GitHub，或只想要一份可照抄的指令清單的維護者。

0. 名詞快速理解

名詞	白話解釋
repo (儲存庫)	專案的資料夾，含所有檔案與歷史紀錄。本專案是 <code>Lycander168/FrankAie-hub</code> 。
clone (複製)	把 GitHub 上的 repo 下載到自己電腦。
branch (分支)	一條獨立的開發線。在分支上改東西不會影響 <code>main</code> 。
commit (提交)	一次存檔，附帶一句說明這次改了什麼。
push (推送)	把本機的 commit 上傳到 GitHub。
pull (拉取)	把 GitHub 上別人的最新變更下載到本機。
PR (Pull Request)	提議把某分支的變更合併進 <code>main</code> ，可供他人審查。

1. 一次性設定 (每台電腦只需做一次)

```
# 設定你的身分 (會顯示在每個 commit 上)
git config --global user.name "你的名字"
git config --global user.email "service@lycander.tw"

# 建議：讓 git 用 main 當預設分支名
git config --global init.defaultBranch main
```

安裝與登入

1. 安裝 [Git](#)。
2. 建議安裝 [GitHub CLI](#) (`gh`) ，登入最方便： `bash gh auth login` 依指示選 `GitHub.com` → `HTTPS` → 用瀏覽器登入即可。

沒有 `gh` 也可以，改用 `HTTPS` + [Personal Access Token](#) 或 `SSH` 金鑰當密碼。

2. 取得專案

```
# 用 gh (推薦)
gh repo clone Lycander168/FrankAie-hub

# 或用一般 git
git clone https://github.com/Lycander168/FrankAie-hub.git

cd FrankAie-hub
```

3. 標準工作流程（每次改東西都照這套）

步驟 1：先同步最新的 main

```
git checkout main
git pull origin main
```

步驟 2：開一條新分支

分支命名建議：`類型/簡短描述`，例如 `feat/supply-chain-expert`、`docs/github-guide`、`fix/roster-typo`。

```
git checkout -b feat/你的功能名稱
```

步驟 3：修改檔案

依需求新增或編輯檔案（例如新增一位專家角色 `skill`，見第 6 節）。

步驟 4：檢視變更

```
git status      # 看哪些檔案被改了
git diff        # 看具體改了哪幾行
```

步驟 5：提交 (commit)

```
git add .                # 把所有變更加入這次提交
git commit -m "新增供應鏈專家 skill (v0.7.0) "
```

好的 commit 訊息：用一句話講清楚「做了什麼」，沿用本 repo 既有風格（中文、必要時附版號），例如： - 新增 4 個原創工具型 skill (v0.3.0) - 團隊結構優化 (v0.5.0) - docs：新增 GitHub 操作指南

步驟 6：推送到 GitHub

```
git push -u origin feat/你的功能名稱
```

-u 只有第一次推送該分支需要；之後同分支直接 git push 即可。

步驟 7：開 Pull Request

```
# 用 gh（會在終端機互動式建立）
gh pr create --base main --title "新增供應鏈專家 skill" --body "說明這次變更內容"
```

或到 GitHub 網站，repo 頁面上會出現 Compare & pull request 按鈕，點下去填標題與說明即可。

步驟 8：合併

審查通過後，在 PR 頁面按 Merge pull request（或 `gh pr merge --squash`）。合併後記得把本機切回 main 並更新：

```
git checkout main
git pull origin main
```

4. 常用指令速查表

想做的事	指令
看目前狀態	<code>git status</code>
看改了哪幾行	<code>git diff</code>
看分支清單	<code>git branch -a</code>
切換分支	<code>git checkout 分支名</code>
開新分支並切過去	<code>git checkout -b 分支名</code>
加入暫存	<code>git add 檔名</code> (或 <code>git add .</code> 全加)
提交	<code>git commit -m "訊息"</code>
推送	<code>git push</code> (首次加 <code>-u origin 分支名</code>)
拉取最新	<code>git pull origin main</code>
看提交歷史	<code>git log --oneline -10</code>
丟棄某檔尚未提交的修改	<code>git checkout -- 檔名</code>
把剛加入暫存的取消	<code>git restore --staged 檔名</code>

5. 常見狀況排解

推送被拒絕 (rejected / non-fast-forward)

遠端有你本機還沒有的提交。先拉取再推：

```
git pull origin 你的分支名 --rebase
git push
```

推送遇到網路錯誤

重試即可；若反覆失敗，間隔 2s → 4s → 8s → 16s 多試幾次。

合併衝突 (merge conflict)

Git 會在衝突檔案中標記 <<<<<<<<、=====、>>>>>>>。手動編輯保留正確內容、刪掉標記，然後：

```
git add 衝突檔名
git commit          # 或 git rebase --continue
```

不小心改錯、想回到乾淨狀態（尚未 commit）

```
git checkout -- .    # 丟棄所有未提交的變更（無法復原，請確認）
```

提交訊息打錯（尚未推送）

```
git commit --amend -m "正確的訊息"
```

6. 本專案專屬：更新 marketplace / 新增角色後

本 repo 是一個 Claude 外掛 marketplace。改完並合併進 `main` 後，使用者端要更新才看得到變更：

```
/plugin marketplace update frankaie-hub
```

新增一位專家角色的完整步驟：

1. 在 `plugins/frankaie-research-hub/skills/` 下新增資料夾，如 `supply-chain-expert/`。
2. 放入 `SKILL.md`，frontmatter 至少要有 `name` 與 `description`。
3. 依第 3 節流程 commit、push、開 PR、合併。
4. 提醒使用者執行上面的 `/plugin marketplace update`。

詳見 `plugins/frankaie-research-hub/README.md`。

7. 黃金守則

1. 永遠不要直接在 `main` 上開發 —— 一律開分支再 PR。
2. 每個 commit 訊息都要看得懂 —— 之後是你自己在查歷史。
3. 推送前先 `git status` 確認 —— 別把不該進版控的檔案（如 `.DS_Store`）加進去。
4. 小步快跑 —— 一個 PR 只做一件事，比較好審查也好回溯。